

Docker Walkthrough

1. What is Docker?

Docker is a platform that allows developers to **build, package, and deploy applications** in lightweight, portable units called **containers**.

It ensures that applications run **consistently** across different environments — whether it's your local system, a testing server, or the cloud.

♦ Key Features

- Lightweight and fast
- Portable across platforms
- Easy to scale and deploy
- Isolates applications using containers

2. What is Containerisation?

Containerisation is the process of **encapsulating an application and its dependencies** (libraries, configurations, runtime, etc.) into a single unit called a **container**.

This approach ensures that the software behaves the same regardless of where it runs — eliminating the “it works on my machine” problem.



Example:

Instead of installing dependencies manually on every system, you can just run:

```
docker run myapp
```

and it works exactly as intended everywhere.

3. What is an Image?

A **Docker Image** is a **read-only template** used to create containers.

It includes everything needed to run an application — the code, runtime, libraries, environment variables, and configuration files.

Think of an **image** as a blueprint, and a **container** as a running instance of that blueprint.

Example:

- Base image: `python:3.9`
- Your app image: built on top of that base image with your source code added.

Build an image using:

```
docker build -t myapp .
```

4. What is DockerHub?

DockerHub is a **cloud-based registry** where Docker users can **store, share, and manage images**.

It's similar to GitHub, but for Docker images.

♦ Key Uses:

- Pull public images (like `ubuntu`, `nginx`, `python`, etc.)
- Push your custom-built images for collaboration or deployment
- Automate builds and integrate with CI/CD pipelines

Example commands:

```
docker pull ubuntu
```

```
docker push username/myapp
```

5. Basic Docker Workflow

1. **Write a Dockerfile** – Define the app's environment and steps to build the image.
2. **Build the Image** – Create a Docker image using your Dockerfile.

```
docker build -t myapp .
```

3. **Run a Container** – Start a container from your image.

```
docker run -p 8080:80 myapp
```

4. **Publish to DockerHub** – Push your image for reuse or deployment.

```
docker push username/myapp
```

6. Example: Simple Python App in Docker

Dockerfile:

```
# Use Python base image
FROM python:3.9

# Set working directory
WORKDIR /app

# Copy files
COPY . /app

# Install dependencies
RUN pip install -r requirements.txt

# Run the app
CMD ["python", "app.py"]
```

Commands:

```
docker build -t python-app .
docker run -p 5000:5000 python-app
```

7. Advantages of Docker

- Consistent environment
- Lightweight and fast startup
- Simplifies CI/CD pipelines
- Easy scaling and version control
- Isolation and security

8. Common Docker Commands

Command	Description
<code>docker build -t name .</code>	Build an image
<code>docker run name</code>	Run a container
<code>docker ps</code>	List running containers
<code>docker images</code>	List available images
<code>docker stop container_id</code>	Stop a container
<code>docker rm container_id</code>	Remove a container
<code>docker rmi image_id</code>	Remove an image
<code>docker pull image_name</code>	Download image from DockerHub
<code>docker push image_name</code>	Upload image to DockerHub